

Atividade Prática: Representação do Mundo de um Jogo Simples

IA para Jogos I - Solução desenvolvida em Python (pygame)

Atividade: Representação do Mundo de um Jogo Simples

Disciplina: Inteligência Artificial para Jogos I — Representação do Mundo Docente: Charles Madeira

O que foi implementado: um nível com 9 áreas em grade, itens coletáveis, inimigos (NPCs) controlados por uma automação simples e uma janela de visualização que não depende da grade.

1. Como executar

O jogo roda localmente com Python 3.10 ou superior:

```
pip install -r requirements.txt
python main.py
```

Os testes do modelo (sem gráficos) também rodam localmente:

```
python -m game.tests          # testes unitários do modelo
python -m game.simulation 20  # análise de balanceamento (20 partidas por cenário)
```

Versão no navegador

Também deixei o jogo publicado em WebAssembly, para testar direto no navegador:

<https://ShinigameBR.github.io/python-simple-world-iaj/>

O empacotamento é feito com o pygbag <https://pypi.org/project/pygbag/>, que converte a aplicação pygame em WebAssembly. Para regerar a versão web:

```
pip install pygbag
pygbag --build --ume_block 0 --title "Simple World" --app_name simple-world .
```

O resultado fica em build/web e é publicado na branch gh-pages do repositório. O main.py roda o loop do jogo com asyncio.run, que é o que o pygbag espera: sem isso o carregamento da página não termina.

2. O que a atividade pede

O cenário é formado por 9 áreas em grade, com itens coletáveis e inimigos. O jogador controla o deslocamento do personagem e os NPCs usam uma técnica simples de automação (no caso, perseguir o jogador em linha reta). A janela de visualização não pode depender da grade: só as áreas próximas ficam ativas e somente os NPCs dessas áreas são atualizados.

As regras do enunciado:

- Uma área vizinha só fica ativa quando o personagem se aproxima da borda;
- No máximo 4 áreas ativas;
- Só os NPCs das áreas ativas são atualizados — perseguem o jogador e causam dano

por contato;

- O jogador coleta primeiros socorros (cura) e TNT (dano em área);
- Qualquer personagem com vida menor ou igual a zero morre;
- O objetivo é sobreviver por um tempo determinado;

- O estado inicial é configurável (JSON ou aleatório) e o balanceamento pode ser

testado em cenários variados.

3. Decisões de projeto

3.1. O mundo como grade regular

Segui a sugestão da aula e modeliei o mundo como uma grade de `Area` (`game/world.py`). Cada área é um retângulo alinhado aos eixos, com limites e centro, e guarda os seus próprios NPCs e itens. A parte importante é que a simulação não precisa percorrer o mundo inteiro: basta saber quais células estão ativas e atualizar o conteúdo delas.

A mesma estrutura também funciona como um grafo de navegação — cada área é um nó e áreas que compartilham uma borda estão conectadas.

3.2. Posição absoluta e pertencimento à célula

As entidades usam coordenadas absolutas do mundo (x , y), mas cada uma pertence a exatamente uma área, calculada por divisão euclidiana das coordenadas pelo tamanho da área (`World.cell_of`). É a forma mais simples de "posicionamento dentro da forma": a área sai da posição, e os testes de colisão ficam baratos. Itens e NPCs ficam na lista da célula (`Area.npcs`, `Area.items`), ou seja, a estrutura funciona como um spatial hash.

Essas listas são dinâmicas. Quando um NPC atravessa a borda de uma área enquanto persegue o jogador, `World._migrate_npcs` o registra na lista da área de destino. Sem isso, um inimigo que tivesse entrado fisicamente numa área ativa continuaria registrado na antiga e "sumiria" sempre que a área antiga fosse desativada, mesmo estando dentro do anel carregado.

3.3. Áreas ativas: anel de carregamento

A reconfiguração dinâmica está em `World.update_active_areas`:

```
1 1
```

```
2 2
```

que formam o anel de carregamento ao redor do jogador;

```
3 3
```

borda da área (zero quando ele está dentro). Qualify-se as que estão a `activation_distance` ou menos;

```
4 4
```

`max_active_areas` (4 por padrão).

Com a distância padrão, que é $1.6 \times \text{area_size}$, o anel fica sempre cheio: a área atual mais até 3 vizinhas mais próximas ficam ativas mesmo quando o jogador está no centro de uma área. Isso evita que os inimigos "desapareçam" quando o jogador caminha para o meio da área ou troca de área — as áreas em volta continuam carregadas, enquanto as distantes ficam congeladas. Com uma distância de ativação curta (50 a 100 px), a regra estrita da atividade aparece: a área vizinha só ativa quando o jogador chega perto da borda.

É a ideia de abstração hierárquica com atualização seletiva, e é o que permite simular mundos com dezenas, centenas ou milhares de NPCs: no máximo 4 áreas têm seus NPCs atualizados a cada iteração.

3.4. Viewport independente da grade

`Viewport` (`game/camera.py`) é uma janela retangular posicionada livremente no mundo (x , y , `width`, `height`), totalmente desvinculada das células: ela pode cortar vários limites de área ao mesmo tempo e não depende das coordenadas da grade. Só as entidades dentro do retângulo visível são desenhadas. A câmera segue o jogador de forma suave (lerp) e é limitada aos limites do mundo.

O jogo abre maximizado em modo janela, ocupando a área de trabalho. Também existe um fator de zoom (`viewport_zoom`, 1.75 por padrão): as coordenadas do mundo são ampliadas na tela, de modo que a janela mostra uma região menor e mais próxima do mundo, cerca de 1,5 áreas de largura. O zoom é só visual — posições,

colisões e a discretização em áreas continuam funcionando em coordenadas do mundo.

O escopo de percepção (áreas ativas da simulação) e a janela visual são coisas independentes, que é justamente a distinção da aula: posição absoluta para a IA, posição relativa ao viewport para a visualização.

3.5. IA dos NPCs: perseguição em linha reta

`NPC.chase(target, dt)` calcula o vetor direção até o jogador, normaliza e avança com velocidade constante. É a automação simples pedida. O dano por contato fica em `contact_damage_to`, que verifica a sobreposição (distância menor que a soma dos raios) e aplica dano por segundo (`contact_damage * dt`). NPCs com vida menor ou igual a zero são removidos em `_cleanup`.

3.6. Itens e efeitos

- Primeiros socorros: na colisão com o jogador, `Player.heal` restaura a

quantidade configurada, respeitando a vida máxima.

- TNT: desenhada como uma caixa com pavio (`_draw_tnt_icon`). Na colisão,

aplica dano em área (`ammo_radius / ammo_damage`) a todos os NPCs dentro do raio, independentemente de a área estar ativa, já que a proximidade é física.

- Feedback da explosão: o modelo emite um evento por acerto

(`World.explosions`) e o renderizador (`Effects`) desenha um preenchimento translúcido que cresce até o raio de dano, mais um anel de contorno fino. Isso deixa visível a área de efeito sem cobrir a cena.

3.7. Simulação em tempo discreto

A simulação avança com `dt` fixo, limitado a $1/120$ no loop do jogo e $1/60$ no modo sem gráficos. É uma simulação discreta com entidades paralelas de forma automática, que é o primeiro modelo da aula. O teste de balanceamento usa o mesmo `World.update`, então os resultados refletem exatamente a mecânica entregue.

3.8. Configuração e geração aleatória

O nível é gerado aleatoriamente a partir de `config.json`, que é a serialização do estado inicial permitida pela atividade: tamanhos, quantidade de NPCs por área, velocidade, dano, distância de ativação, tempo de sobrevivência e a seed. A seed garante que os experimentos sejam reproduzíveis.

3.9. Simulação sem gráficos para balanceamento

`game/simulation.py` roda o modelo sem pygame, sob uma política simples e determinística de jogador (gulosa: procurar cura quando estiver ferido, senão procurar TNT). Ele avalia 12 cenários com `N` seeds e gera o relatório de balanceamento. É o item "testar o balanceamento em cenários diferentes" feito como experimento reproduzível, em vez de jogo manual.

4. Organização do código

<code>main.py</code>	– entrada do pygame: loop, input, estados, HUD, render
<code>config.json</code>	– estado inicial do nível / parâmetros
<code>game/</code>	
<code>entities.py</code>	– Player, NPC, Item (movimento, vida, efeitos)
<code>world.py</code>	– Area e World (grade, áreas ativas, atualização seletiva)
<code>camera.py</code>	– Viewport (janela visual independente)
<code>simulation.py</code>	– análise de balanceamento (<code>python -m game.simulation</code>)
<code>tests.py</code>	– testes do modelo (<code>python -m game.tests</code>)
<code>smoke_test.py</code>	– teste de renderização/screenshot (sem janela)
<code>docs/</code>	
<code>solucao.md</code>	– este documento
<code>balance_report.md</code>	– relatório de balanceamento gerado
<code>screenshots/</code>	– screenshots (menu, jogo, pausa)

Controles

Tecla	Função
WASD / Setas	Mover
P	Pausar
R	Reiniciar o nível
ENTER	Começar (no menu)
ESC / Q	Menu / Sair

5. Relação com os conceitos da disciplina

Conceito da aula	Onde aparece
Grade regular como abstração do mundo	Grade de 9 Areas dividindo o mundo em células do mesmo tamanho
Posicionamento dentro da célula	<code>cell_of()</code> calcula a área a partir das coordenadas absolutas
Lista de entidades por célula	<code>Area.npcs</code> / <code>Area.items</code>
Posição absoluta × relativa (viewport)	Viewport converte mundo → tela; a IA usa coordenadas absolutas
Escopo de percepção (limitar consultas)	<code>update_active_areas()</code> , anel de carregamento, no máximo 4 áreas
Atualização seletiva / hierarquia	Só os NPCs de áreas ativas executam chase e dano por contato
Simulação discreta	Loop com <code>dt</code> fixo
Automação de NPC (perseguição direta)	<code>NPC.chase</code>
Grade como grafo	Áreas vizinhas ficam conectadas implicitamente
Árvores/quadtrees (extensão)	Evolução natural do particionamento (seção 7)

6. Resultados de balanceamento

Gerados por `python -m game.simulation 20`, disponíveis em `docs/balance_report.md`. São 20 partidas por cenário, com a política gulosa, e 90 s de sobrevivência salvo quando indicado.

Cenário	Inimigos	Sobrevivência	Vida média	Abates	Morte (s)	Áreas ativas (média)
1. Base (3x3, 600px)	4–9 por área	50%	45.8	52	75.7	4.0
2. Dezenas de inimigos	3–5 por área	70%	66.3	33	80.3	4.0
3. Centenas de inimigos	40–60 por área	0%	0.0	46	3.4	4.0
4. Milhares de inimigos	300–500 por área	0%	0.0	28	0.5	4.0
5. Áreas pequenas (300px)	4–9 por área	100%	98.5	59	—	4.0
6. Áreas grandes (900px)	4–9 por área	30%	21.0	35	57.8	3.95

Cenário	Inimigos	Sobrevivência	Vida média	Abates	Morte (s)	Áreas ativas (média)
7. Ativação curta (100px)	4–9 por área	100%	95.4	53	—	1.23
8. Ativação longa (500px)	4–9 por área	65%	60.9	52	72.9	3.32
9. NPC rápido (220)	4–9 por área	15%	15.0	21	5.6	4.0
10. Contato letal (60/s)	4–9 por área	25%	23.9	32	22.5	4.0
11. Sobreviver 30s	4–9 por área	100%	84.8	52	—	4.0
12. Sobreviver 180s	4–9 por área	40%	37.2	52	80.9	4.0

Leitura dos resultados

O anel de carregamento padrão mantém as 4 áreas do redor ativas quase o tempo todo (média 4.0), então os inimigos não somem perto do jogador e as áreas distantes ficam congeladas. Como os NPCs são remanejados quando atravessam a borda (`_migrate_npcs`), quem entra numa área ativa continua sendo atualizado e atacando, o que torna os encontros no anel persistentes em vez de intermitentes.

Na escala, multiplicar a quantidade de inimigos multiplica a multidão dentro do anel carregado: nos cenários 3 e 4 a política gulosa morre antes de juntar TNT suficiente. Os milhares de inimigos distantes existem, mas não são simulados enquanto a área deles não entra no anel, que é justamente o objetivo da atualização seletiva.

Sobre o tamanho das áreas: áreas pequenas aumentam os confrontos com a mesma densidade. Áreas grandes diluem os NPCs, mas como o anel cobre uma região física maior, mais inimigos ficam ativos ao mesmo tempo, e o cenário fica mais difícil (30%).

A distância de ativação controla bem o comportamento: com raio curto a regra estrita de "perto da borda" emerge, com média de 1.23 área ativa e 100% de sobrevivência; com raio longo o comportamento se aproxima do anel completo (65%).

Velocidade do NPC e dano por contato são os fatores que mais pesam na dificuldade (15% e 25% de sobrevivência). São os parâmetros que eu mexeria primeiro para ajustar o nível.

Com o mapa padrão e um jogador passivo, a sobrevivência fica em 50%: a política gulosa limpa o anel com TNT, mas os NPCs que atravessam a borda continuam chegando. Um jogador humano que se move e usa o espaço tem mais folga.

Configuração final: considerando o jogador humano, deixei o `config.json` com velocidade do jogador 260, NPC 150, dano de 25/s, cura de +40, raio de TNT 220 e 90 s de sobrevivência. É uma base equilibrada para se jogar manualmente.

7. Extensões possíveis

- Carregamento e descarregamento dinâmico: o mecanismo de áreas ativas já

desativa áreas distantes e congela os NPCs delas. Em mundos bem maiores que 3×3, as listas poderiam também ser descarregadas da memória e recriadas na ativação, que é o gancho natural em `World.update_active_areas`.

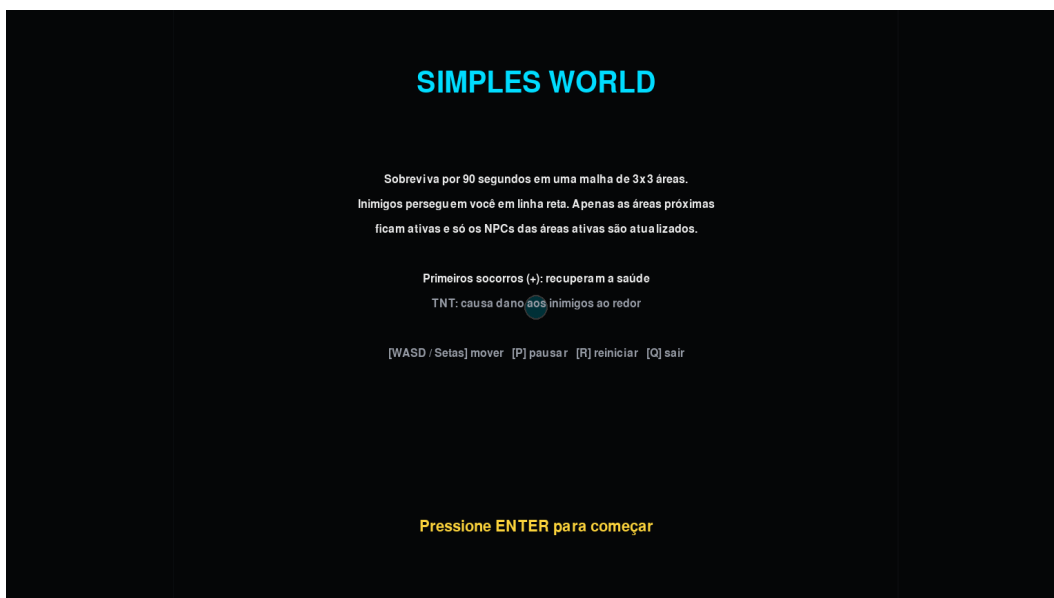
- Quadtree: trocar a grade fixa por uma Quadtree, seguindo a aula, mantendo a ideia de listas de entidades por região.

- Busca de caminho: como a grade é um grafo, o chase poderia usar BFS na vizinhança para contornar obstáculos.

- Curva de dificuldade: variar distância de ativação e velocidade ao longo da partida usando os mesmos campos do config.json.

O relatório de balanceamento, os testes e os screenshots podem ser regerados com os comandos da seção 1.

Apêndice - Screenshots da solução



Tela inicial (menu)



Jogando: viewport, áreas ativas e entidades



Tela pausada